

COMP1009

Programming Paradigms

Lecture 12

Iterator Pattern

Last lecture

- Generics – add a parameter type, T

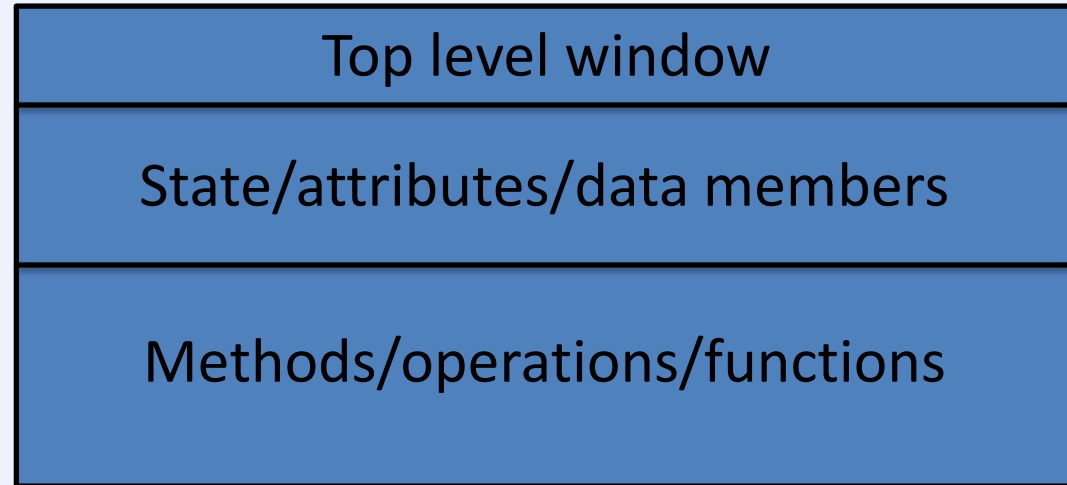
```
public class GenArray<T>
{
    private int size = 0;
    private Object[] data = new Object[0];
```
- Problems of type erasure
 - Don't know type at runtime, so can't create object of unknown type T, or array of type T[]
 - Solution: treat it as base class type
- Wrapper classes for basic types
 - Auto-boxing

This Lecture

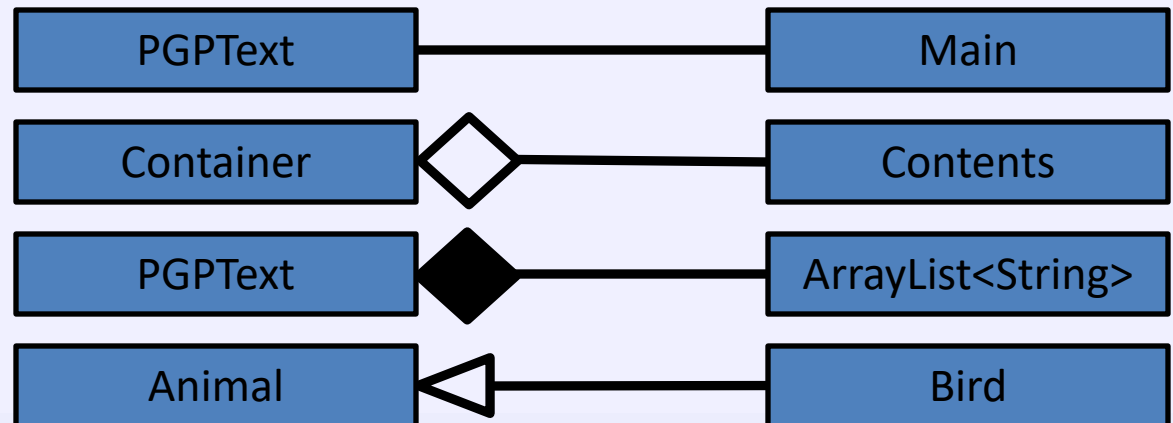
- Class diagrams and inner classes
- Some comments about collection classes
 - Generic classes (parametric polymorphism)
 - Powerful ways to store/manipulate data
 - Many things done for you already
 - We focus here on OO concepts, not Java class library!
 - But be happy with ArrayList as it is a useful example of generics
- Iterator / Iterable
- A linked list class

Class diagrams - from lecture 6

- Box for each class
 - Three sections:
 - Class name
 - Member data
 - Methods/functions



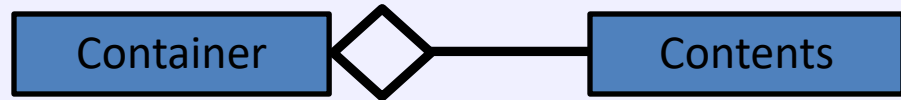
- Line for each relationship – symbol-end matters!
 - Association
 - Aggregation
 - Composition
 - Is-A / type of



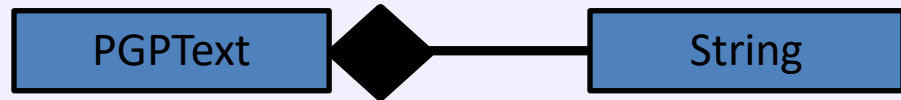
'Oddity' in class diagram – inner class

- Has-A (aggregation/composition) – from lecture 6

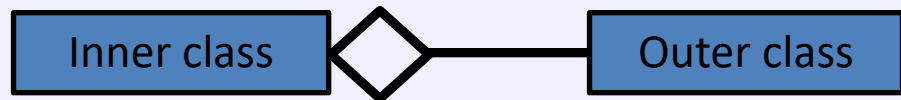
- Container has contents



- PGPText has Strings



- Inner class object has an outer class object associated with it
 - It knows which outer class object is associated with it, and can access it
- Outer class object doesn't have an easy way to know what inner class objects are associated with it
- i.e., the inner class object HAS-A outer class object, not the other way around



- Unintuitive at first! Defining a class within another class doesn't make the outer object 'contain' or 'have' inner class objects, instead inner object has 'has-an' outer object

Two more comments on generics

- **Can you subclass a generic class?**

- Yes – either keeping the generic type...

```
public class GenericSubclass<T> extends GenArray<T>
```

- Or using a specific type

```
public class GenericSubclass2 extends GenArray<Integer>
```

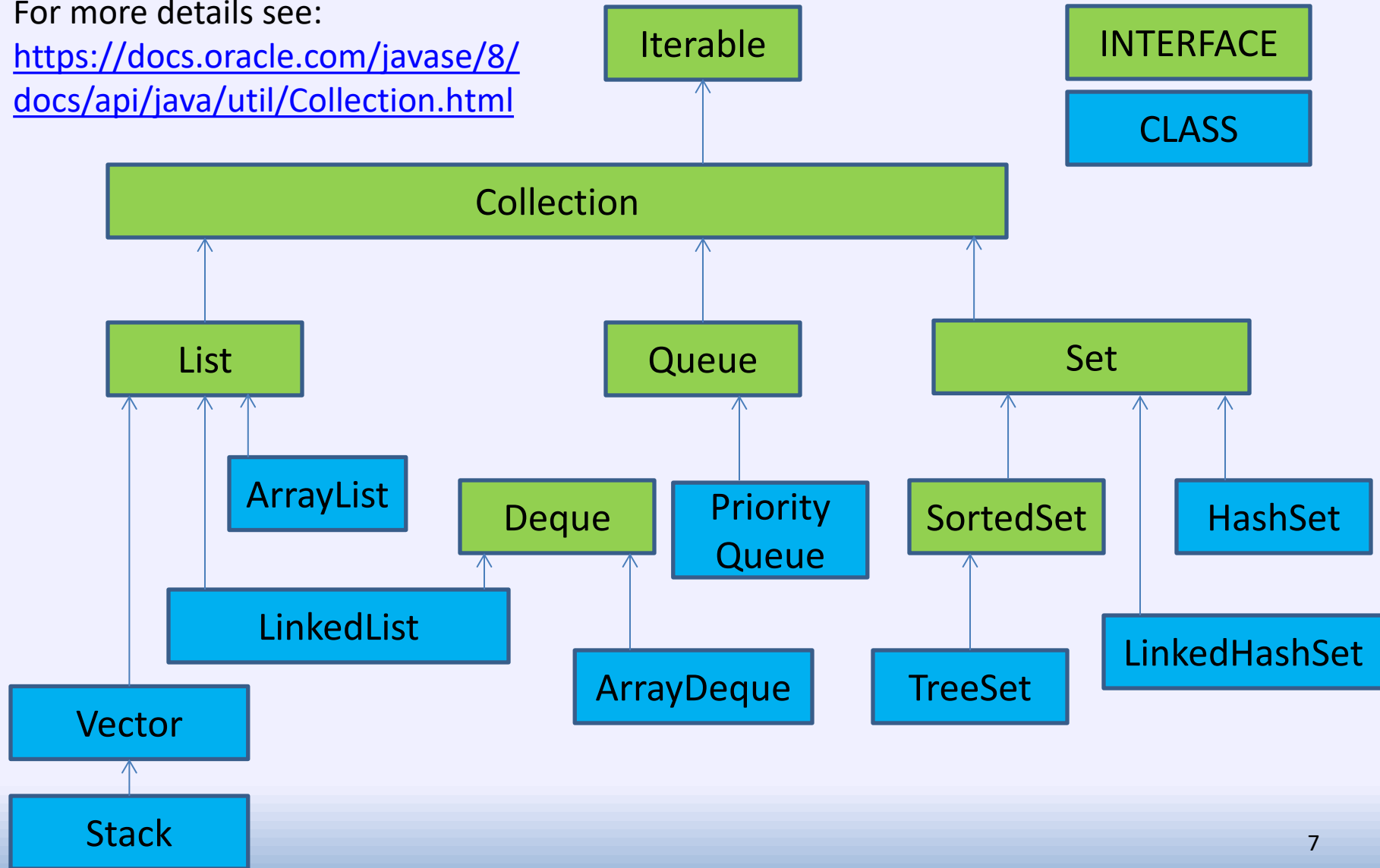
- **Can you have generic interfaces? Yes**

```
public interface TestInterface<T>
{
    T get();
    void set( T value );
}
```

Java's collection classes are generics

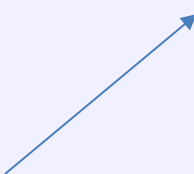
For more details see:

<https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html>



Collection Class Usage Example

```
ArrayList<String> list = new ArrayList<>();  
list.add("Item 1");  
list.add("Item 4");  
list.add("Item 3");  
list.add("Item 2");  
list.addAll( list );  
list.sort( null/*default comparator*/ );  
for( String str : list )  
    System.out.println( str );
```



We can do this. Why (how) it works is the topic of today's lecture

Aside: Algorithms and Collections class

```
Collections.shuffle(list);
```

```
System.out.println( "Shuffled list: ");
```

```
for( String str : list ) System.out.println( str );
```

```
LinkedList<String> list2 = new LinkedList<>();
```

```
list2.addAll(list);
```

```
List<String> values = Collections.nCopies( 3, "z" );
```

```
list2.addAll(values);
```

```
list2.addAll( Collections.nCopies( 3, "a" ) );
```

```
System.out.println( "New list: ");
```

```
for( String str : list2 ) System.out.println( str );
```

```
Collections.sort(list2);
```

```
System.out.println( "Sorted new list: ");
```

```
for( String str : list2 ) System.out.println( str );
```

```
Collections.reverse(list2);
```

Example you could try

More comments on the collections

- Importantly these actually sort/reverse/etc the collections, they do not create new copies
 - In FP immutable data means we get/need a new copy
- Collections have many other features, e.g. `binarySearch()`, `frequency()`, `indexOfSubList()`
- Actually, when we cover Lambdas in Lecture 13, we see that there is a lot more we can do really easily:

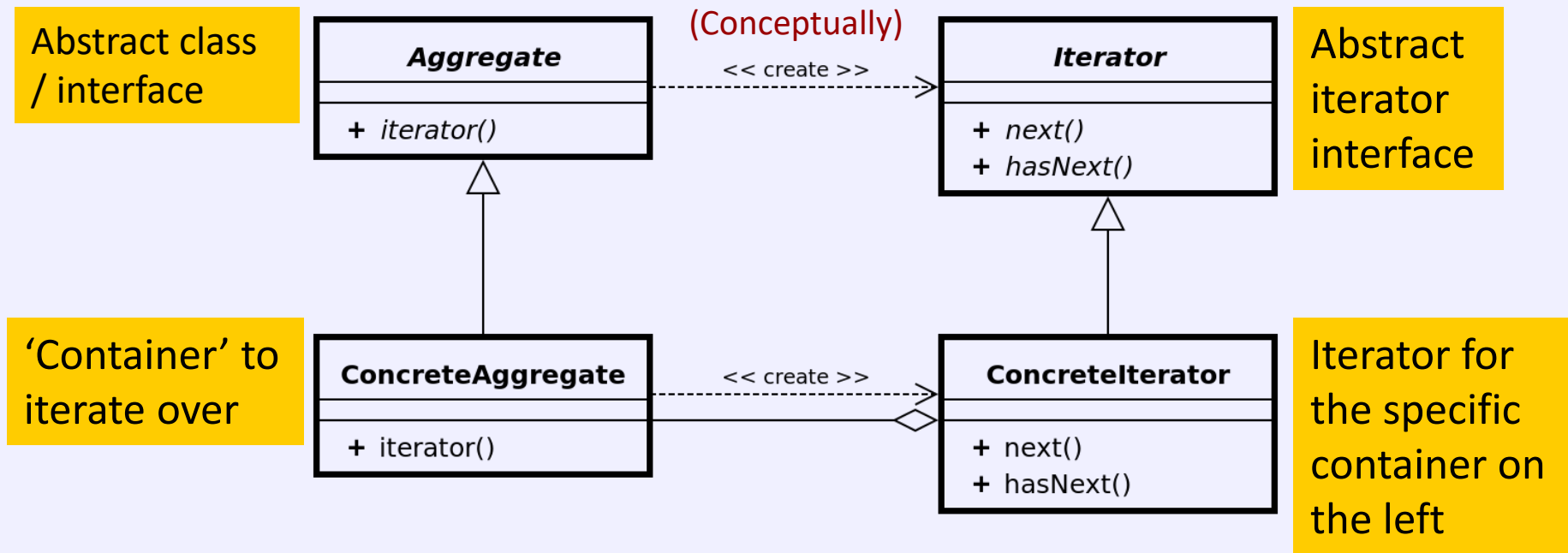
```
Collections.sort( list2,  
    (a,b)-> a.compareToIgnoreCase(b) );  
Collections.sort( list2,  
    (a,b)-> -a.compareToIgnoreCase(b) );
```
- And we can very easily create/add more functions
- Note: in general functions like sort will change the order of the collection itself, not create a copy (Haskell copies)

Iterator Pattern

Used in for-each loops:
`for( String str : list2 )`
Do something with str

Design Pattern : Iterator

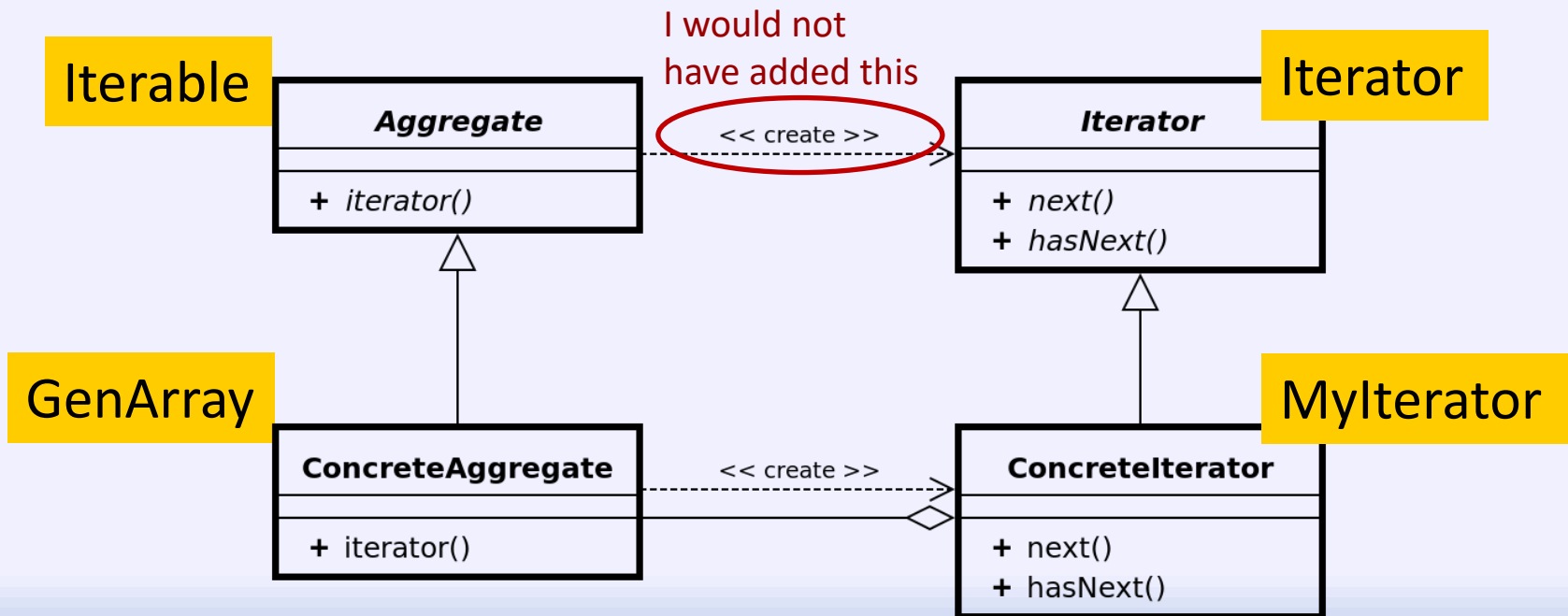
- A method to move through (iterate through) the elements of something
 - Retrieve an Iterator, which remembers its position and allows you to move to the next element



- Aside: arrow labelled `<<create>>` means aggregate creates iterator
- `<<create>>` is a descriptive annotation

Java Classes for the Iterator pattern

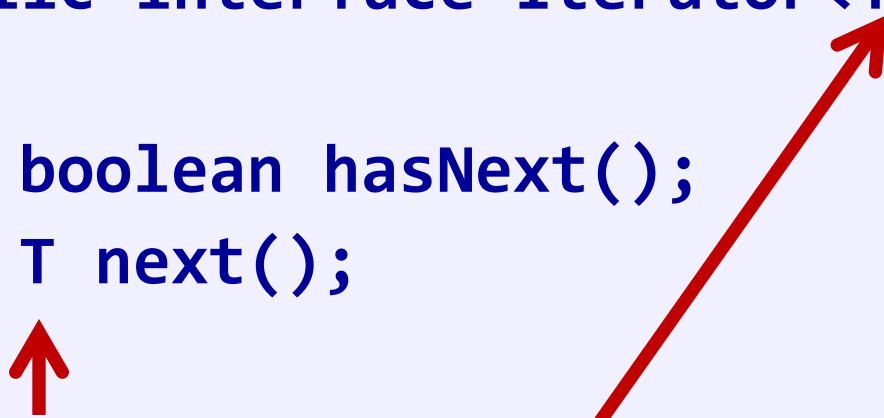
- Iterable and Iterator interfaces already exist in Java, built-in
- You create the sub-classes and implement the methods
- You need an iterator to iterate through it (e.g. MyIterator)
- Your container/aggregate (e.g. GenArray) must be able to create an object of the correct iterator type (via the iterator() method)



Java support for Iterator and Iterable

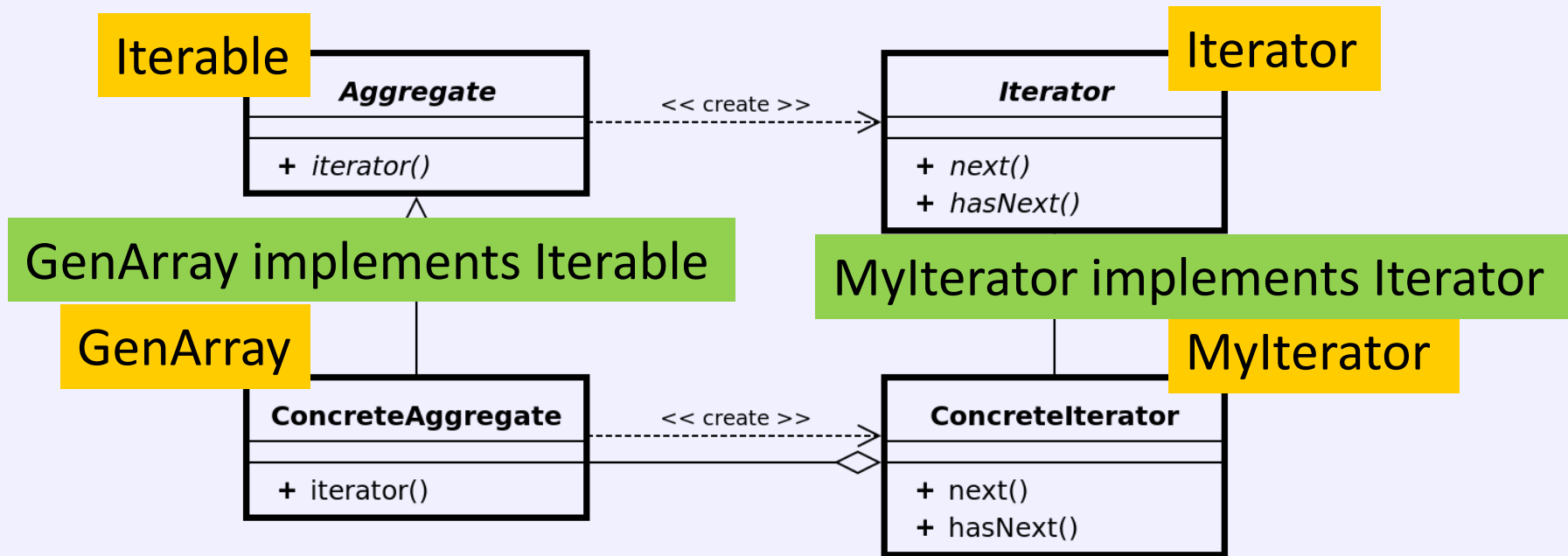
- Standard Iterator interface exists in Java:

```
public interface Iterator<T>
{
    boolean hasNext();
    T next();
}
```



- Note the generic type here, on the Iterator and next()!
- Also note that there are two other functions, which you can ignore (remove() and forEachRemaining())
- Iterable interface also exists, to create an Iterator
- Java ‘for each loops’ can be used with Iterators
 - for (type name : iterable) ...

Our objects implement the interfaces



Java Support for Iterators

- Implement `Iterable<T>` on the Collection class

implements Iterable<T>

- This means that you need to support the method:

@Override

public Iterator<T> iterator()

{

return new MyIterator();

}

- We will create an inner class for the Iterator, and return an object of that type

Iterator works like this...

- Using the generic array from last lecture:

```
GenArray<Integer> arr = new GenArray<Integer>();  
arr.add(1); arr.add(2); arr.add(3);
```

- Get an iterator, and keep retrieving the next entry while there is one...

```
Iterator<Integer> it = arr.iterator();  
while( it.hasNext() )  
    System.out.println( it.next() );
```

- The following code does the same thing:

```
for ( Integer i : arr ) // for each loop  
    System.out.println( i );
```

There are two interfaces to know

- **Iterable** (on the container to iterate through)
 - Interface with a method to create an iterator
Iterator<T> iterator()
- **Iterator** (object to do the iteration)
 - Interface to allow:
 - checking whether there is a next object (hasNext),
 - and, if so, retrieving it and moving on (next)**boolean hasNext()**
T next()

We can support **Iterable** interface and create an **Iterator** object

```
public class GenArray<T> implements Iterable<T>
{ ...
```

```
    class MyIterator implements Iterator<T>
    {
```

```
        int index = 0;
```

```
        @Override
```

```
        public boolean hasNext()
        { return index < data.length; }
```

```
        @Override
```

```
        public T next()
        { return get(index++); }
```

```
    }
```

```
    @Override
```

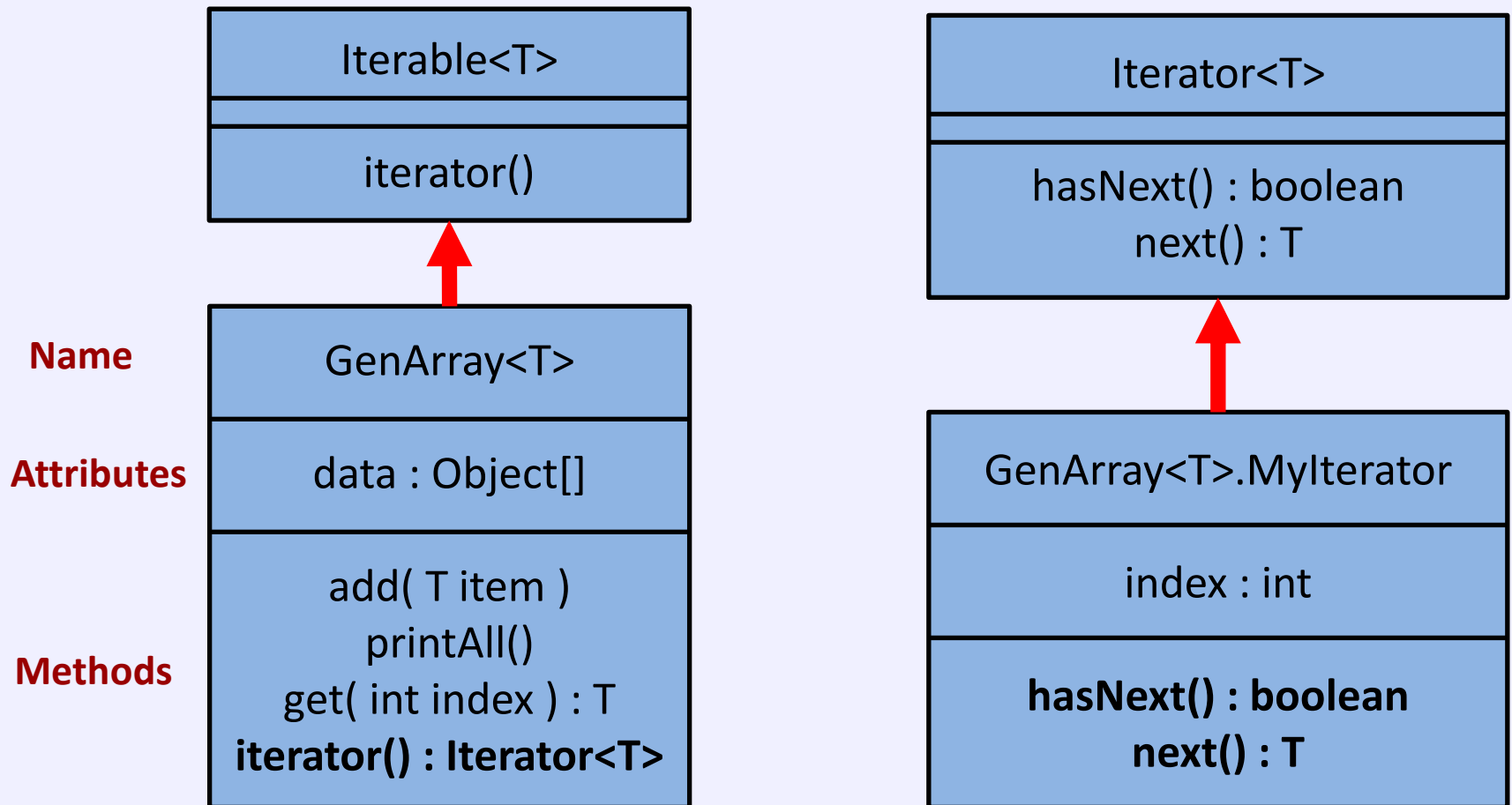
```
    public Iterator<T> iterator()
    { return new MyIterator(); }
```

Hint for Eclipse:
Implement the interface
and let it create the methods

hasNext(): Is there a
current one to get?

next(): Get current one then
move to point to next one

Simplified Class Diagram : iterators



- Note: I massively simplified the class diagrams, to make them easier to understand and be closer to Java format
- In any exam we check for the main concepts

Two ways to use the iterator

```
public static void main(String[] args)
{
    GenArray<String> arr2 = new GenArray<String>();

    // Note: we don't care what type of object it
    // iterates, we just use the correct iterator
    Iterator<String> it2 = arr2.iterator();
    while( it2.hasNext() )
        System.out.println("" + it2.next() );

    for ( String str : arr2 )
        System.out.println("" + str );
}
```

List class

A linked list class (full)

```
public class LinkedList
{
    public class ListItem
    {
        private int value = 0;
        private ListItem next = null;

        public int getVal() { return value; }
        public ListItem next() { return next; }

        // Create a new item
        public ListItem( int val, ListItem next )
        {
            this.value = val;
            this.next = next;
        }
    }

    // Head of list
    private ListItem head = null;

    // Add new item to head of the list
    public void add( int i )
    {
        head = new ListItem( i, head );
    }

    // Print everything in the list
    public void printAll()
    {
        recursivePrint( head );
        System.out.println();
    }

    // Print the rest of the list, then current item
    public void recursivePrint( ListItem current )
    {
        if ( current == null )
            return; // Nothing to do
        recursivePrint(current.next());
        System.out.print(" "+current.getVal());
    }
}
```

Next few slides show you the details

A linked list class (part 1)

```
public class LinkedList
{
    public class ListItem
    {
        private int value = 0;
        private ListItem next = null;

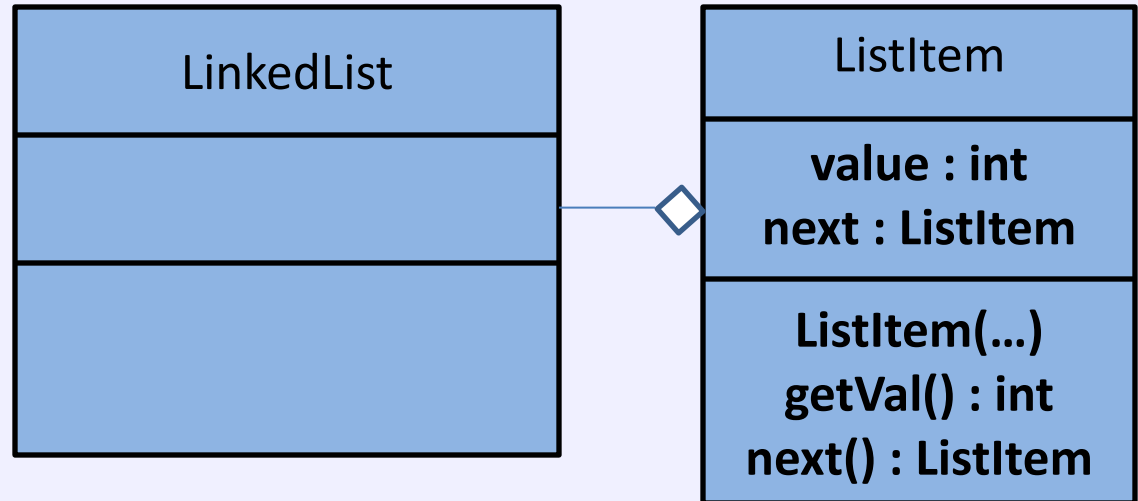
        public int getVal() { return value; }
        public ListItem next() { return next; }

        // Create a new item
        public ListItem( int val, ListItem next )
        {
            this.value = val;
            this.next = next;
        }
    }
}
```

ListItem is an Inner class
(Could be static at the moment,
a nested class,
but later we will want an inner class)



Simplified Class Diagram (1)



- Remember: **ListItem** has-a **LinkedList** that it is in
 - Counter-intuitively, usually done as aggregation where inner class ‘has a’ outer class
- **ListItem** knows which **LinkedList** it is for – has one
- **LinkedList** has many **ListItems**, and no direct access

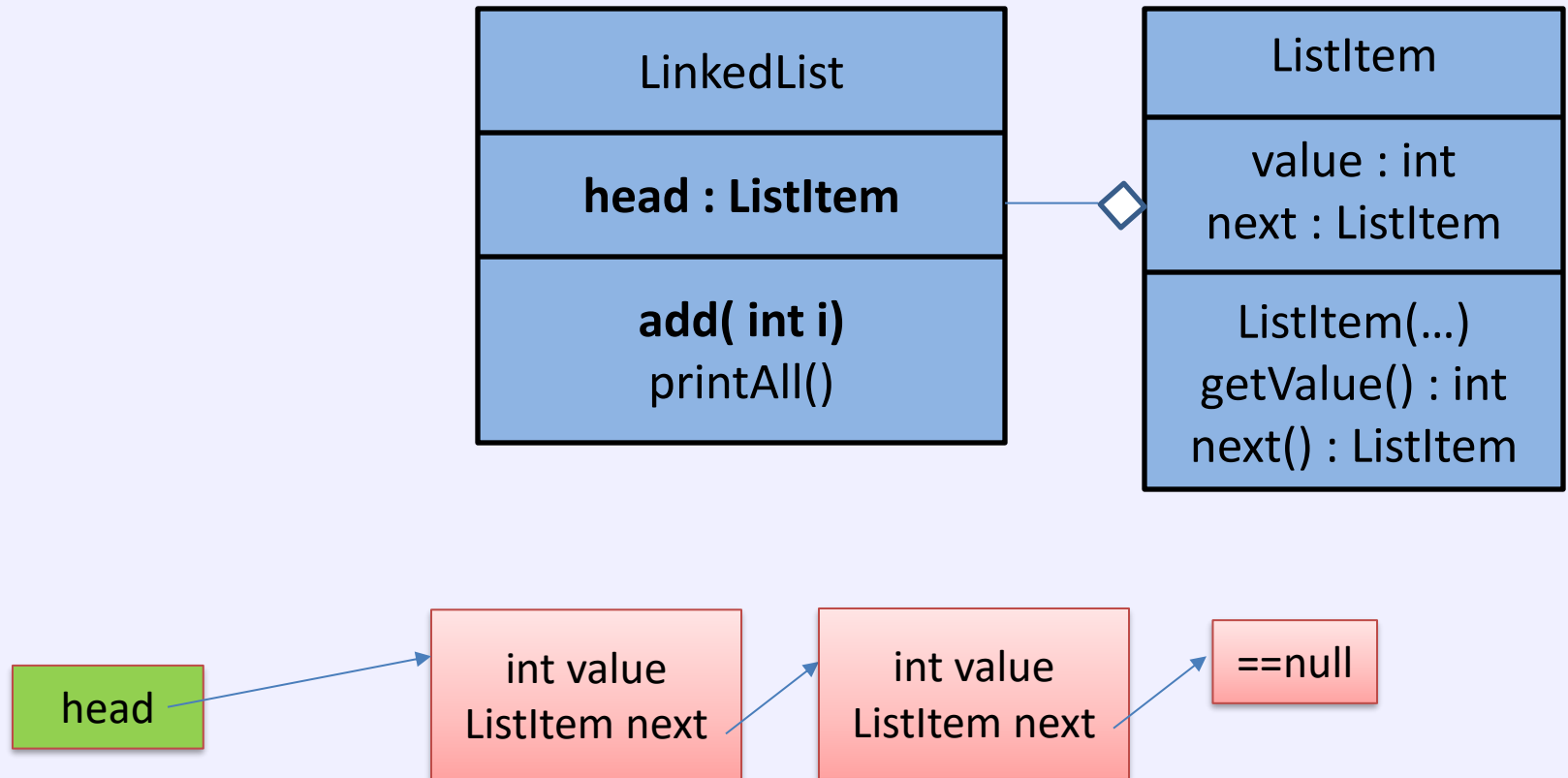
A linked list class (part 2)

```
public class LinkedList
{
    public class ListItem
    {
        public int getVal() { ... }
        public ListItem next() {... }
        public ListItem( int val, ListItem next ) { ... }
    }

    // Head of list
    private ListItem head = null;

    // Add new item to head of the list
    public void add( int i )
    {
        head = new ListItem( i, head );
    }
}
```

Simplified Class Diagram (2)



- LinkedList has members in it
- One attribute (head of list), two methods

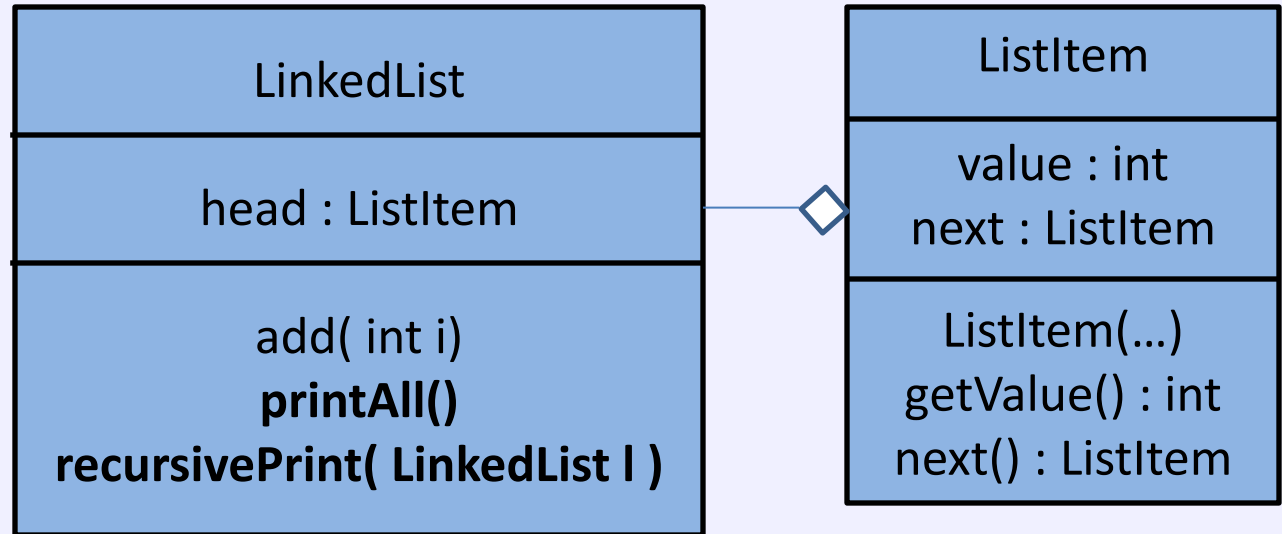
A linked list class (part 3)

```
public class ListItem
{
    public int getVal() { ... }
    public ListItem next() {... }
}
private ListItem head = null;
```

```
void printAll()
{ // Print everything in the list
    recursivePrint( head );
    System.out.println();
}

void recursivePrint( ListItem current )
{ // Print the rest of the list, then current item
    if ( current == null )
        return; // Nothing to do
    recursivePrint(current.next());
    System.out.print(" "+current.getVal());
}
```

Simplified Class Diagram (3)



- **recursivePrint()** prints the rest of the list, then the current item – so order gets reversed, last first
- Warning: stack space!

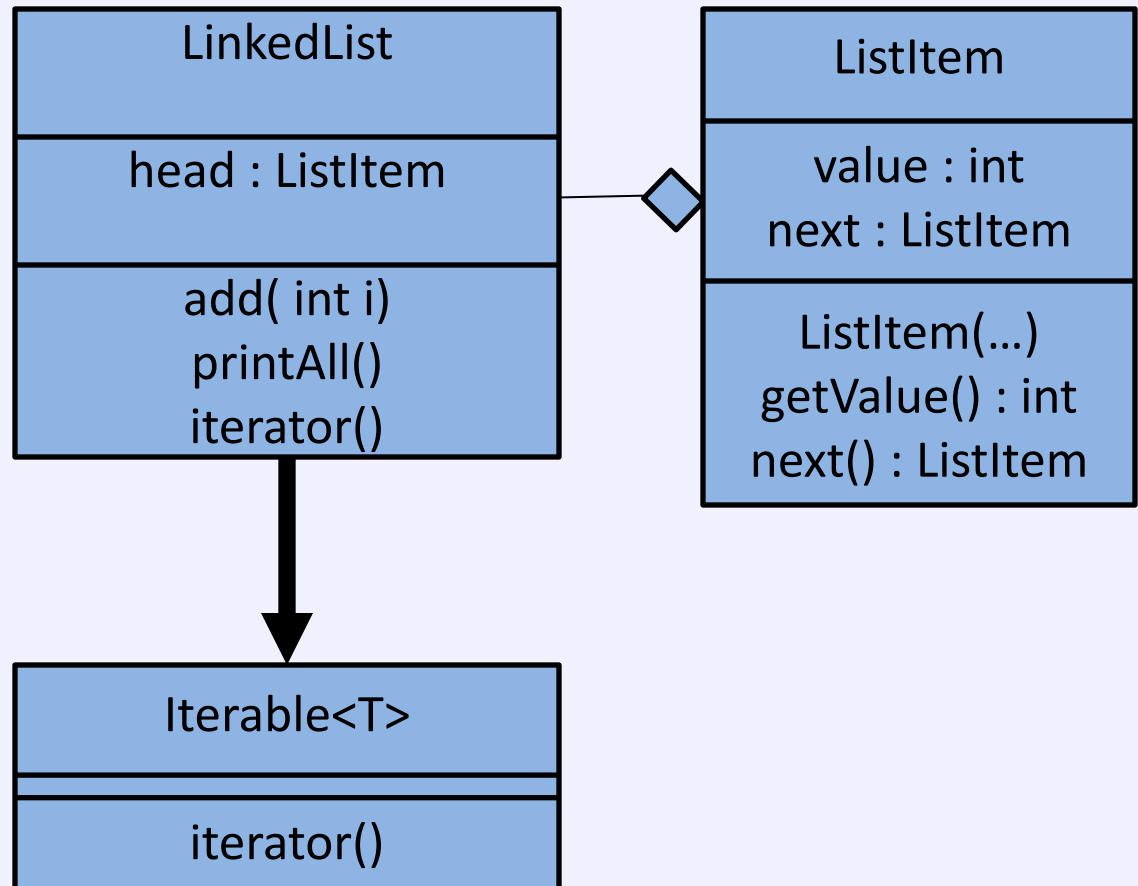
A main function for List

```
public class MainTest
{
    public static void main(String[] args)
    {
        LinkedList lst = new LinkedList();
        lst.printAll();
        lst.add(1);
        lst.printAll();
        lst.add(2);
        lst.add(4);
        lst.printAll();
    }
}
```

Generic Linked List Class

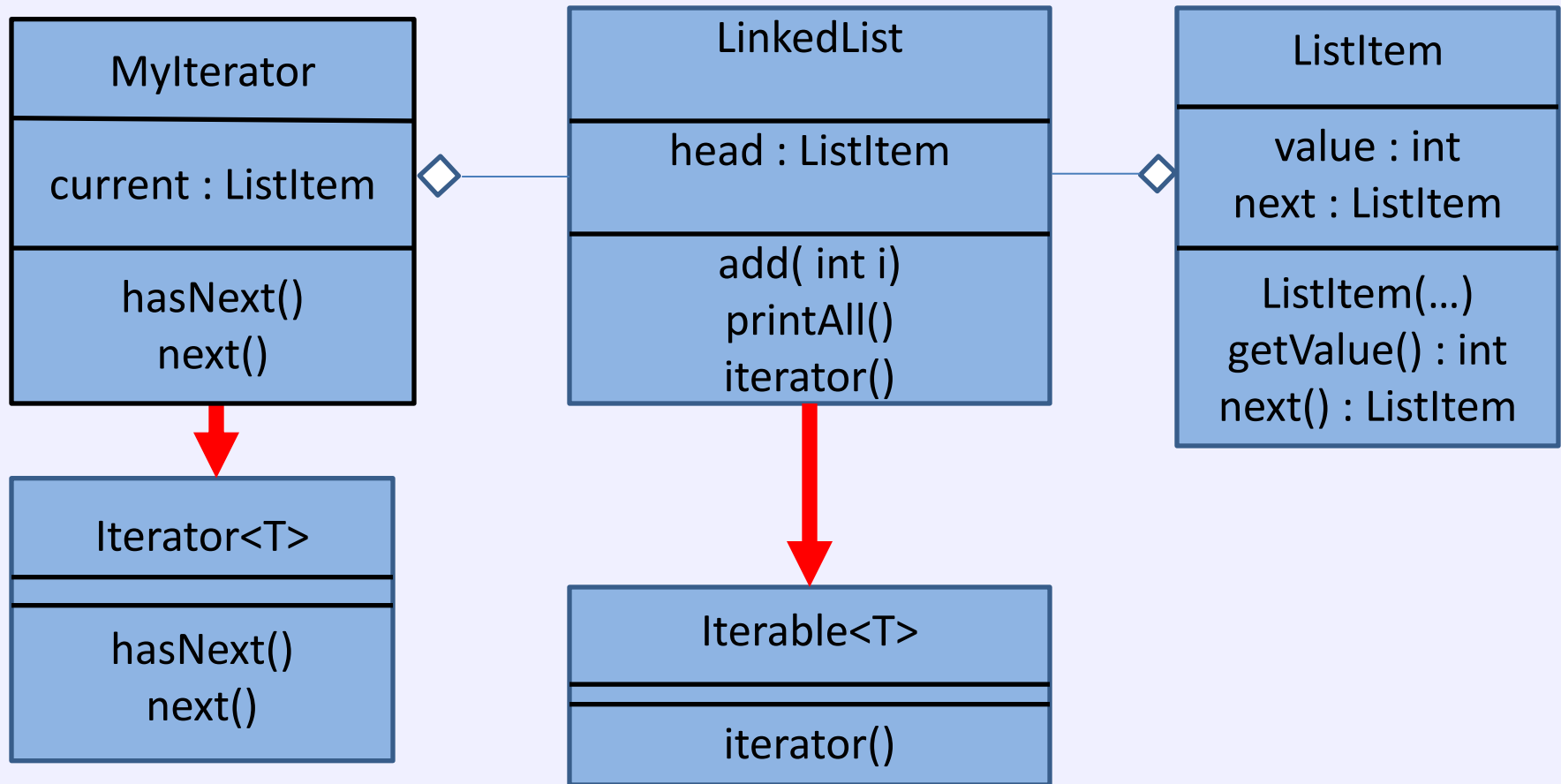
- We can make this class a generic class
- Add <T> to class name
- Change the content type from int to T
- We can add also an Iterator to this class
 - In the same way as we did for the array

Simplified Class Diagram (4)



- Implement Iterable

Simplified Class Diagram (5)



- Create another inner class for MyIterator

The full code for the the iterator

```
@Override
public Iterator<T> iterator()
{
    return new MyIterator();
}

public class MyIterator implements Iterator<T>
{
    ListItem current = head;

    @Override
    public boolean hasNext()
    {
        return current != null;
    }

    @Override
    public T next()
    {
        T t = current.getVal();
        current = current.next();
        return t;
    }
}
```

Using the list

- You can use the generic list and have access to the for-each loop (for(x : y) format)

```
MyIterator<String> list3  
    = new MyIterator<String>();  
list3.add("First");  
list3.add("Second");  
for ( String s : list3 )  
    System.out.print( s );
```

Next Lecture

- Anonymous classes and Lambdas